

✅✅ DAY 10 — MEMORY + CONTEXT (YOUR TEMPLATE VERSION)

✅✅ WHAT YOU ADD TODAY

Planner (Day 9) + ChatMemory (already exists ✅) = Conversational Agent ✅🔥

✅✅ STEP 1 — USE EXISTING MEMORY

📌 use: memory/chat_memory.py ✅

✅ Your existing code (KEEP SAME)

```
from collections import deque

class ChatMemory:

    def init(self, size=10):

        self.memory = deque(maxlen=size)

    def add(self, user, response):

        self.memory.append({ "user": user, "response": response })

    def get(self):

        return list(self.memory)
```

✅✅ STEP 2 — UPDATE agent.py

📌 File: app/agent.py

✅ Modify imports

```
from memory.chat_memory import ChatMemory # ✅ CHANGED

from app.planner import Planner

from models.llm import generate
```

✅ Update class init

```
class Agent:
```

```
def init(self, rag_system, excel_tool=None):

    self.rag = rag_system

    self.excel = excel_tool

    self.planner = Planner()

    self.memory = ChatMemory() # ✅ USING YOUR MEMORY
```

✅ Replace FULL run() with this

```
def run(self, query):

    # ✅ Step 1: get past conversation

    history = self.memory.get()

    # ✅ convert memory → text

    past_context = ""

    for item in history:

        past_context += f"User: {item['user']}\nAssistant: {item['response']}\n"

    # ✅ enrich query

    enriched_query = f"""

Conversation so far:

{past_context}

Current question:

{query} """

    # ✅ Step 2: planning

    plan = self.planner.create_plan(enriched_query)

    if not plan:

        plan = [("EXCEL", query)]

    print("FINAL PLAN:", plan)

    results = []
```

✅ Step 3: execute plan

for i, (tool, task) in enumerate(plan, 1):

print(f"Executing Step {i}: {tool} → {task}")

if tool == "EXCEL" and self.excel:

result = self.excel.smart_query(task, generate)

elif tool == "RAG":

docs = self.rag.search(task)

result = "\n".join(docs[:3])

else:

result = "Unknown tool"

print(f"STEP {i} RESULT:", result)

results.append(f"Step {i} Result:\n{result}")

✅ Step 4: final answer WITH memory

final_prompt = f"""

Conversation:

{past_context}

Results:

{results}

User question:

{query}

Give final answer:

"""

final_answer = generate(final_prompt)

✅ Step 5: store memory

self.memory.add(query, final_answer)

return final_answer

✅✅ STEP 3 — RUN

```
python -m app.main
```

✅✅ TEST LIKE THIS

```
>> what is average price
```

```
>> what about year 2015
```

```
>> and lowest km?
```

✅✅ RESULT

✅ remembers previous question

✅ supports follow-up queries

✅ multi-step + memory together ✅🔥

✅✅ WHAT YOU JUST BUILT

Before: ❌ Stateless multi-step agent

Now:

✅ Conversational multi-step agent

✅ Uses history

✅ Understands context

✅✅ YOUR SYSTEM NOW

✓ RAG ✅

✓ Excel Agent ✅

✓ Tool Routing ✅

✓ Multi-step Planner ✅

✓ Memory (ChatMemory) ✅🔥
